

Sajed Atwa

Csc 44700- Introduction to Machine Learning

Professor: Erik Grimmerman

November 20, 2024

## **Sentiment Analysis on Movie Reviews Using Logistic Regression, Naive Bayes, and LSTM**

The history of movie reviews traces back to the earliest days of cinema. Initially critics would write essays in newspapers and magazines and would focus the artistic merit and directional styles of movies. The modern concepts of film critiquing started taking off around 1940s. Prominent movie critics like James Agee and Pauline Kael helped to bring more mainstream attention to movie reviews and helped to establish a style of review that aimed to persuade people to watch the movies with insightful analysis of the film. The introduction of television shows made film criticism even more popular by making it even more accessible to a wider audience of people, which emphasized how enjoyable a film was to watch and using more accessible rating systems. The current rating system made by the Motion Pictures Association of America (MPAA) that provides parental guidance on movie content, emerged in 1968, when the MPAA chairman Jack Valenti replaced the earlier moral censorship guidelines, which was known as the Hays Code. The current system was made from the notion that the movie industry should not control what an audience sees, but rather educate parents on the contents of the film to help them make decisions on what to watch as a family. This introduced descriptors like PG, PG-13, and R that rate for which ages the content in the movie is suitable for, these descriptors apply to every rated film. With new these ratings, parents are now able to decide whether a movie is

appropriate to watch with their kids. Today movie reviews are widely available online and not only consider a critics in-depth thoughts and analysis, but what the audience thought of the film.

With the vast amount of user-generated content that is available online due to the digital era, it creates tons of opportunities for researchers and businesses to extract insights from textual data. Sentimental analysis, which is also known as opinion mining, is critical for understanding the public's opinion by classifying text automatically into positive, negative, or neutral. It can be used for many applications from analyzing customer feedback on products, to gauging public opinion on political campaigns. For this project I used sentiment analysis on movie reviews from the IMDB dataset. This would be useful to help gauge the public's overall opinion on movies. Movie reviews can be challenging however due to how subjective they could be, and how often figurative language is used, sarcasm, and mixed sentiment. In order to tackle this I used three machine learning models: Logistic Regression, Naïve Bayes, and Long Short-Term Memory (LSTM). The Logistic Regression and Naïve Bayes model serve as interpretable baseline models, while the Long Short-Term Memory (LSTM) model is being used due to its ability to context models, and because it has sequential dependencies.

The purpose of this project is to evaluate the success of these models' abilities to predict sentiment, by keeping tracks of numerous metrics such as accuracy and feature importance. These metrics will help provide useful insights on how successful and effective these models are for classifying sentiment in these movie reviews. By comparing the metrics and results of these 3 different models, we will be able to highlight the trade-offs between the simplicity, computational cost, and performance of these models when it comes to analyzing sentiment in movie reviews.

For this project I ended up using the IMDB Large Movie Review Dataset. This dataset is used for binary sentiment classification, and it contains a vast amount more data than the previous benchmark datasets. This dataset is a widely used benchmark for sentiment analysis, and it contains 50,000 labeled reviews, and it is evenly split between positive and negative sentiments. With this balance we can ensure there will be no bias to a particular sentiment when we use this data to train and test and models. The reviews in this dataset range from a few sentences to detailed multi-paragraph analyses, which makes it a rich and diverse dataset for model training.

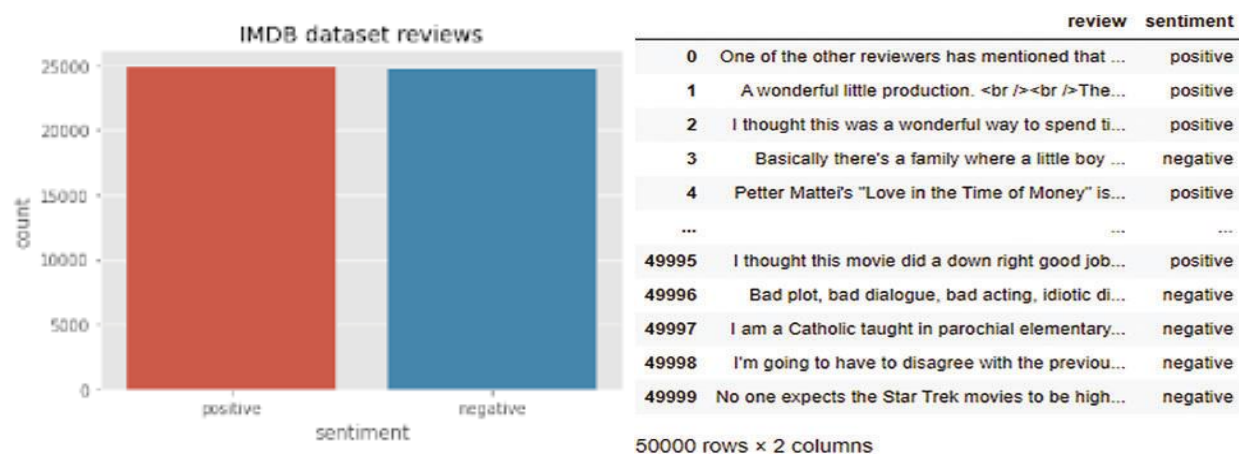


Figure 1: IMDB Large Movie Review Dataset

To prepare the dataset for use in the machine learning models, I took some preprocessing steps which is crucial for preparing the raw reviews for model training. Some of the text preprocessing steps I took are Text Cleaning. Text Cleaning includes converting all the text in the dataset to lowercase, removing all punctuation from the datasets, and also removing stop words such as “the”, “and”, and “is”, as these do not contribute in any way to the sentiment of a review. Another step I took was label encoding, which means I encoded the sentiments as 1 for positive reviews, and 0 for negative reviews. Next, I prepared the dataset for feature extraction. The Logistic Regression and Naïve Bayes models used TF-IDF (Term Frequency-Inverse

Document Frequency) vectorization. This approach helps to quantify the importance of a word in a document relative to the entire text. This will create numerical representations of text that have a vocabulary size of 5000 words. The Long Short-Term Memory (LSTM) model however used tokenization instead. Tokenization will convert all the text into sequences of integers, along with padding to ensure each of the sequences are of the same length, which is 200 words. These preprocessing steps helped to standardize the dataset across all three of the models, while also using their unique strengths.

Doing sentiment analysis on user-generated content such as movie reviews is by nature very difficult. Some of the reasons include the noise the movie reviews contain. Movie reviews may often contain typos, informal language, and irrelevant information. This makes it difficult for machine learning models to focus on any meaningful patterns. Another reason is movie reviews are often very ambiguous. Some phrases and words can have different sentiments and meanings based on the context they are placed in. An example is the phrase “not bad”, has a positive sentiment, despite containing the word “bad” which is usually negative. Another reason is the different lengths of each review. The reviews can vary in length from single sentences to lengthy paragraphs, which means the models are required to be able to handle both short and long texts effectively. There may also be challenges specific to different models. For example, the Logistic Regression and Naïve Bayes models rely on bag-of-words representations. These representations ignore word order, which limits these models’ ability to understand context. These models also may struggle with rare words or phrases that may carry strong sentiments. The Long Short-Term Memory (LSTM) model, however, often requires a vast number of computational resources due to its recurrent architecture. The model is also often susceptible to overfitting without the use of proper regularization techniques such as dropout. Another

challenge in sentiment analysis is overfitting. Overfitting occurs when a machine learning model learns about patterns that is specific to the training data its given but fails to generalize on new and unseen data. To prevent this from occurring, the Logistic Regression and Naïve Bayes use regularization parameter to avoid the models becoming overly complex. The Long Short-Term Memory (LSTM) model will employ dropout layers, which means it will randomly exclude some input, to help reduce the risk of overfitting. Another Issue that may occur with sentiment analysis is data leakage, however this will be mitigated by ensuring proper train-test splits without overlapping data are used.

The first machine learning model I implemented into this dataset was the Logistic Regression model. The Logistic Regression machine learning model is a fundamental algorithm that is widely used for tasks that require binary classification, such as sentiment analysis. Although Logistic Regression is a very simple model, it remains an effective baseline model due to how efficient it is at computing and how interpretable it is. The way this algorithm works is by finding a linear relationship between the features and the output classes. In our case the features would be the word vectors, and the output classes would be the positive or negative sentiments. The sigmoid function also known as the logistic function, is what maps the linear combination of the features to probability values between 0 and 1. These probability values represent the likelihood of an input belonging to one of the classes. If the probability value exceeds a certain value, which is usually set at 0.5, then the model will classify the review as positive. If the probability value is below 0.5 then the review will be classified as negative.

In order to implement the Logistic Regression model, I needed to follow several key steps. The first step I did was to convert all the reviews to lowercase, remove all punctuation and stop words. Doing this made sure only words that are meaningful would be used to classify the

reviews. The next step I did was applying TF-IDF vectorization. I applied the TF-IDF vectorization with a vocabulary size of 5000 words. Limiting the vocabulary size to 5000 words would help to reduce the complexity of the computations, while maintaining the terms which were most informative. I then split the dataset into 80 percent training data and 20 percent testing data to evaluate how the model would perform on any new or unseen reviews. I used the LogisticRegression class from the scikit-learn library to implement the model. I kept all the default parameters and used L2 regularization to prevent overfitting. Then I used the training data that I transformed using the TF-IDF vectorization to train the model and made predictions on the test set. The model was able to be trained in seconds, which shows how efficient the algorithm is at computing things.

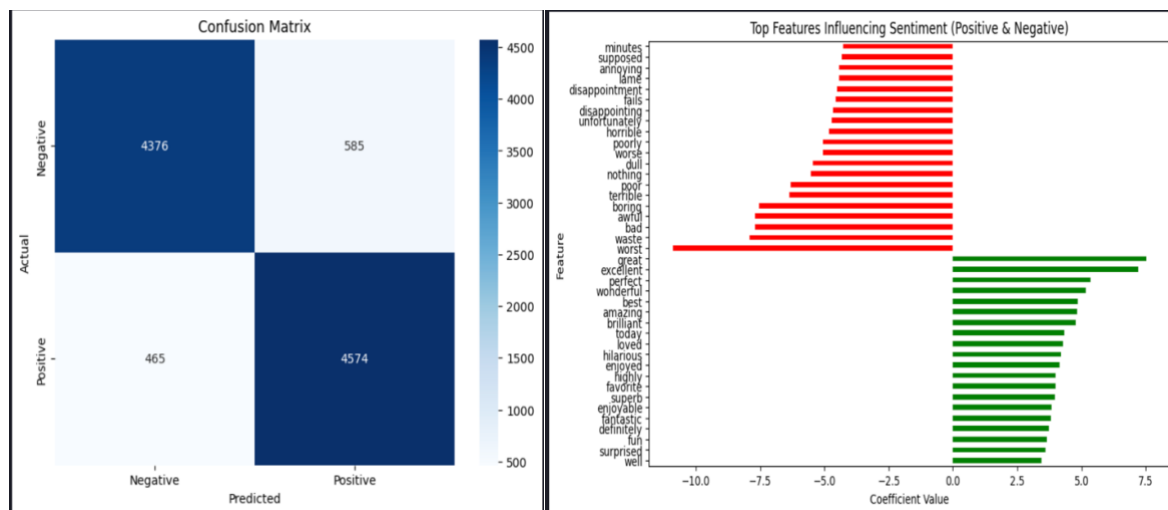


Figure 2: Logistic Regression Model Results

Above are the graphs generated based on the predictive performance of the Logistic Regression model. The model achieved an accuracy of 89.5 percent on the test set. This indicates that the model correctly classified the sentiment of 89.5 percent of the reviews in the test set. I generated a confusion matrix, which is shown in figure 2, to further analyze the performance of the Logistic Regression model. By analyzing the confusion matrix, it is revealed that from the

test set, 4574 reviews were correctly classified as positive, 4376 reviews were correctly classified as negative, 585 negative reviews were misclassified as positive, and 465 positive reviews were misclassified as negative. I also calculated the model's precision, recall, and F1-scores, which was 0.90 precision, 0.88 recall, and 0.89 F1-score for negative sentiment, and 0.89 precision, recall 0.91, and 0.90 F1-score for positive sentiment. These values being very close to each other shows that there was a balanced performance for both positive and negative sentiments for the model. The model overall performed very well overall; however, it struggled to classify reviews that contained mixed sentiments, or subtle language nuances. This led to some of the reviews being misclassified.

By observing the learned weights associated with each of the features we can see what words most influenced the model's predictions. By observing the Top features graph in figure 2, we can see that words such as "amazing", "great", and "fantastic" were given high positive weights, while words such as "terrible", "boring", and "worst" were given high negative weights. These show these words strong association with their respective sentiment. This highlights the model's reliance on explicit sentiment words, which highlights the model's inability to handle complex linguistic structures like sarcasm and context-dependent phrases. Phrases like "not bad" may be misclassified as negative due to containing the word "bad".

The Logistic Regression model was a strong baseline for sentiment analysis. Its strengths were its computational efficiency, simplicity, and its interpretability. It is very quick to train the model and very easy to extract insights from however, its reliance on a bag of words representation, which ignores word order and context, make it unable to effectively classify reviews that require contextual understanding or involve sarcasm.

The next machine learning model I implemented into this dataset was the Naïve Bayes model. The Naïve Bayes model is a probabilistic algorithm which is based on the Bayes Theorem. It is also very widely used for tasks that require classification of texts, such as sentiment analysis, due to how simple and efficient it is. This model assumes that features such as words in a document, are conditionally independent given what class label its given, which simplifies the computations needed for probabilities. Even though it has this strong assumption, the model still performs tasks that involve little and high-dimensional data, such as text classification, very well. The way this algorithm works is by calculating the posterior probably of each of the classes, in this case positive and negative sentiments, given the features it inputs. When classifying new texts, the Naïve Bayes model predicts the class with the highest posterior probability, which is calculated using the formula  $P(C|X) = (P(X|C)P(C))/P(X)$ , where  $P(C|X)$  is the posterior of the class C given the features X,  $P(X|C)$  is the likelihood of the features given the class,  $P(C)$  is the prior probability of the class, and  $P(X)$  is the probability of the features. Using this formula the model is able to efficiently classify text into the sentiment categories.

In order to implement the Naïve Bayes machine learning model, I had to complete some necessary steps. Similarly to the Logistic Regression model, I preprocessed the reviews in the dataset by converting everything to lower case, then removed all punctuation and stop words. I then applied TF-IDF vectorization to the dataset and limited the size of the vocabulary to the 5000 most frequent words. Doing this would help balance the model's complexity and computational efficiency. Then I split the dataset into 80 percent training data and 20 percent testing data to test the model. This model was implemented using the MultinomialNB class from the scikit-learn library. While the model was being trained, it calculated the prior probabilities of each class, which in this case is the positive and negative sentiments, and the likelihood of each



of the words were given a class label. The model then used these probabilities to classify the reviews in the test set.

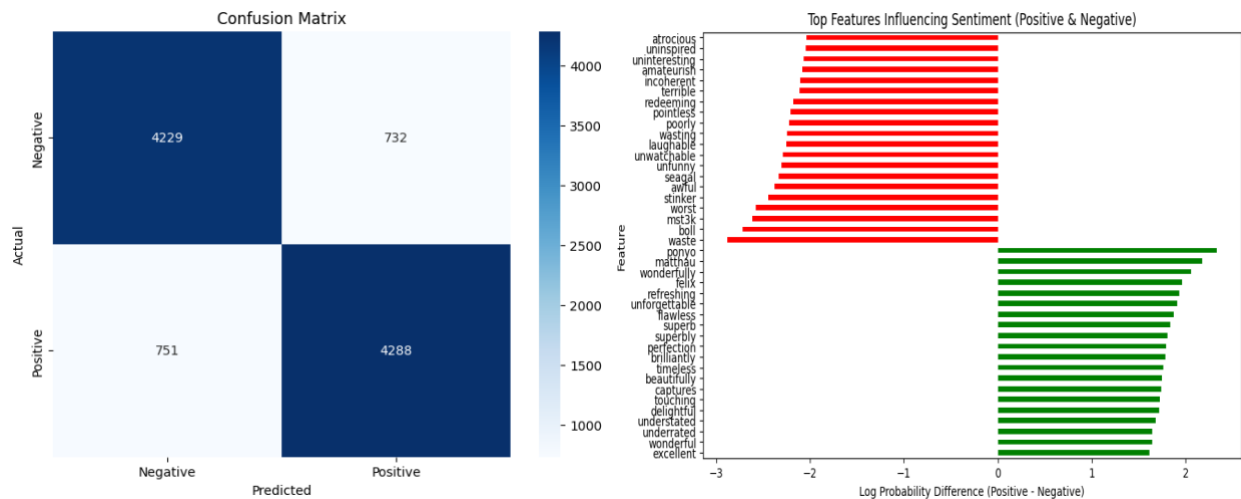


Figure 3: Naïve Bayes Model Results

Above are the graphs generated based on the predictive performance of the Naïve Bayes model. The model achieved an accuracy of 85.17 percent on the test set. This means that the model correctly classified the sentiment of 85.17 percent of the reviews in the test set. I generated a confusion matrix, which is shown in figure 3, to further analyze the performance of the Naïve Bayes model. By analyzing the confusion matrix, it is revealed that from the test set, 4288 reviews were correctly classified as positive, 4229 reviews were correctly classified as negative, 732 negative reviews were misclassified as positive, and 751 positive reviews were misclassified as negative. I also calculated the model's precision, recall, and F1-scores, which was 0.85 precision, 0.85 recall, and 0.85 F1-score for negative sentiment, and 0.85 precision, recall 0.85, and 0.85 F1-score for positive sentiment. These values being identical to each other shows that there was a balanced performance for both positive and negative sentiments for the model. This model performed very well on strongly polarized reviews; however, it struggled in classifying in ambiguous reviews and reviews that contained very subtle sentiment changes. It

also struggled in classifying reviews with rare words, and reviews with complex sentence structures. This resulted in some reviews being misclassified.

By examining the log probabilities of the words given each class, we can determine which words had the strongest influence with positive and negative sentiments. By examining the top features graph in figure 3, we can see that words such as “unforgettable”, “flawless”, and “wonderfully” had strong association with positive reviews, while words like “waste”, “worst”, and “stinker” had a strong association with negative reviews. This shows that like the Logistic Regression model, the Naïve Bayes model also had a major reliance on explicit Sentiment words when classifying reviews. Although the model performed well overall, its assumption of feature independence may lead to misclassifications of reviews in cases where word combinations or context are important, for example, the phrase “not good” maybe be classified as positive due to the presence of the word “good”.

The Naïve Bayes model is a strong and practical choice for text classification tasks and has many advantages for sentiment analysis. These advantages are that its computationally efficient so it’s perfect for large datasets like IMDB, it performs well for little text data, and that its probabilistic nature makes it good for straightforward interpretation of predictions. However, the model’s assumption of conditional independence between features makes it unable to classify complex textual data effectively. It also, like the Logistic Regression model, does not account for word order and context, which is needed for understanding any nuanced sentiments.

The final machine learning model I implemented was the Long Short-Term Memory (LSTM) model. LSTM networks are a type of Recurrent Neural Network (RNN) specifically designed to handle sequential data and learn long-term dependencies. Unlike traditional RNNs, which struggle with the vanishing gradient problem, LSTMs utilize memory cells and gates to

selectively retain and forget information over time. By using these gates (the forget gate, input gate, and output gate), the model can determine which information to keep, which to discard, and how to update the hidden state. In sentiment analysis, LSTMs excel at understanding the order and context of words in a sentence. This ability makes LSTMs perfect for determining the sentiment of textual data, where it's meaning often depends on word order and context.

I implemented the LSTM model by using the TensorFlow Keras Library. The first step I did was tokenize all the textual data. This means each review was split into a sequence of words and mapped to an integer index. Then to ensure each input size was consistent, I padded all the sequences to a length of 200 words. The architecture of the LSTM consisted of these 3 layers: Embedding layer, LSTM layer, and Dense Output layer. For the embedding layer I set the input dimensions (vocabulary size) to 5000 and the output dimension to 128, mapping each word to a dense vector representation. For the LSTM layer, it was a single layer with 128 units added, along with a dropout rate of 0.2 to prevent overfitting. The recurrent dropout was also set to 0.2, ensuring regularization across time steps. The Dense Output Layer was a connected layer with a sigmoid activation function was used to output a probability for the binary classification task. I trained the model for 5 epochs with a batch size of 64 using the Adam optimizer and binary cross-entropy loss function. I split the dataset into 80 percent training data and 20 percent testing data to test the model.

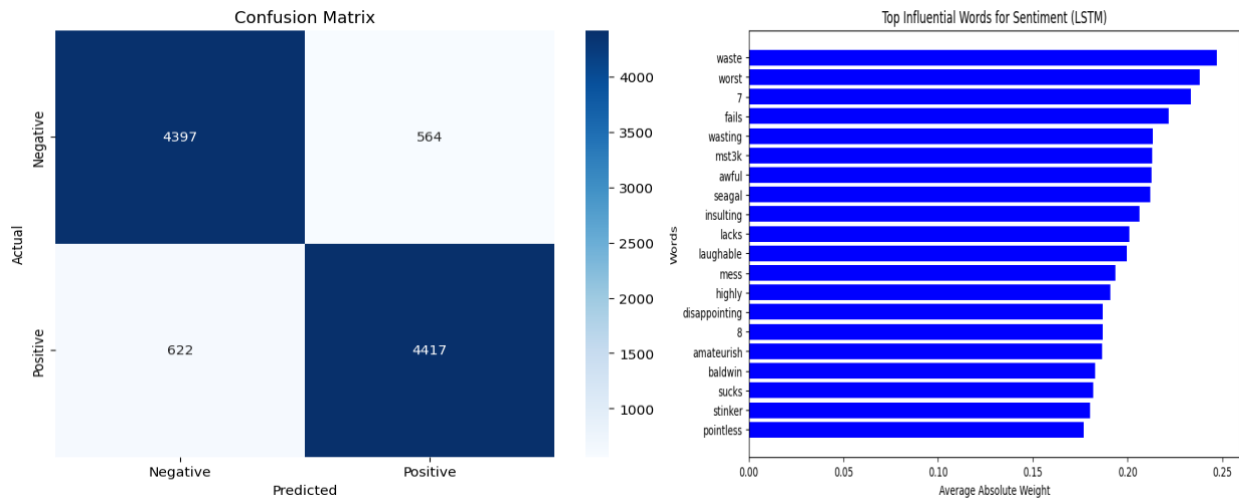


Figure 4: Long Short-Term Memory (LSTM) Model Results

Above are the graphs generated based on the predictive performance of the LSTM model. The model achieved an accuracy of 88 percent on the test set. This means that the model correctly classified the sentiment of 88 percent of the reviews in the test set. I generated a confusion matrix, which is shown in figure 4, to further analyze the performance of the LSTM model. By analyzing the confusion matrix, it is revealed that from the test set, 4417 reviews were correctly classified as positive, 4397 reviews were correctly classified as negative, 564 negative reviews were misclassified as positive, and 622 positive reviews were misclassified as negative. I also calculated the model's precision, recall, and F1-scores, which was 0.88 precision, 0.89 recall, and 0.88 F1-score for negative sentiment, and 0.89 precision, recall 0.88, and 0.88 F1-score for positive sentiment. These values being like each other shows that there was a balanced performance for both positive and negative sentiments for the model. These results highlight the LTSM's ability to capture the complex patterns in the textual data, especially in cases where word order and context played an important role. The model's robustness in differentiating between the sentiments is shown by the high number of correct classifications.

By examining the embedding layer of the LSTM model we can see the weights of each of the words. By examining the top features graph in figure 4, we can see that words such as "excellent," "brilliant," and "fantastic" had strong association with positive reviews, while words like "terrible," "awful," and "poor" had a strong association with negative reviews. This shows that the LSTM has the ability learn nuanced representations of words based on their context in the training data. However, interpreting these weights is less straightforward compared to simpler models, as the embeddings capture multi-dimensional relationships between words.

The LSTM model's strength is its ability to understand sequential dependencies and contextual information. This makes it very effective for sentiment analysis tasks involving complex sentences or phrases. Unlike Logistic Regression and Naive Bayes, which rely on individual word frequencies, LSTMs can understand the relationships between words over long distances in a sentence. However, this all comes at a cost as LSTMs are computationally expensive, so they require a lot more time and resources to train compared to traditional models. They also lack the interpretability of Logistic Regression and Naive Bayes, as their predictions are based on intricate patterns learned in the embedding and recurrent layers.

The Logistic Regression, Naive Bayes, and LSTM models all had varying performances, highlighting the trade-offs between simplicity, computational cost, and predictive accuracy. Logistic Regression achieved an accuracy of 89.5%, making it the most efficient baseline model due to its quick training time and easy interpretability. It performed well on reviews with explicit sentiment words but struggled with contextual phrases and mixed sentiments, such as sarcasm or nuanced language. Similarly, Naive Bayes achieved an accuracy of 85.17%, showing competitive results for strongly polarized reviews but faced challenges with ambiguous text and rare words. Its reliance on the independence assumption limited its ability to handle context-

dependent phrases. The LSTM model, with an accuracy of 88%, outperformed both models in capturing sequential dependencies and nuanced word relationships. It excelled in reviews requiring contextual understanding but was more computationally expensive and less interpretable due to its complex architecture. Overall, while the baseline models provided strong starting points, the LSTM's ability to understand textual context demonstrated its superiority for more complex sentiment analysis tasks.

For this project I explored and compared the performance of three machine learning models, Logistic Regression, Naive Bayes, and LSTM, for sentiment analysis on movie reviews. The Logistic Regression model stood out for its simplicity and computational efficiency, achieving the highest accuracy among the models. Naive Bayes provided a fast and interpretable solution, particularly excelling in reviews with explicit sentiment indicators, but struggled with nuanced language due to its independence assumption. The LSTM model demonstrated superior ability to understand context and sequential dependencies, making it well-suited for complex text classification tasks. However, its computational cost and lack of interpretability remain notable challenges. Some future work that could be done would be to use additional deep learning models like transformers (e.g., BERT) and use more epochs when running the model to increase its accuracy. These results provide a foundation for advancing sentiment analysis approaches and expanding their applications across various domains.

## References

History of Movie Reviews:

<https://www.filmratings.com/History#:~:text=The%20current%20rating%20system%20emerged,%2C%20R%20and%20NC%2D17.>

Dataset:

<https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

LSTM information:

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Naïve Bayes information:

<https://www.datacamp.com/tutorial/naive-bayes-scikit-learn>

Logistic Regression information:

[https://scikit-learn.org/dev/modules/generated/sklearn.linear\\_model.LogisticRegression.html](https://scikit-learn.org/dev/modules/generated/sklearn.linear_model.LogisticRegression.html)

Code for LSTM model:

<https://youtu.be/C5wRAILuuY0?si=HeLgJjgiXtol9GxF>